

Topic 9: Polymorphism and Virtual Functions

9.1 What is Polymorphism?

- Polymorphism means 'many forms' -- the same function call produces different behaviour depending on the actual object type
- Compile-time polymorphism: function overloading, operator overloading (resolved at compile time)
- Runtime polymorphism: virtual functions (resolved at runtime through dynamic dispatch)
- Runtime polymorphism is the core OOP mechanism that makes extensible designs possible

9.2 Virtual Functions

- Mark a base class function virtual to allow derived classes to override it
- When called through a base class pointer/reference, the correct derived version runs at runtime
- Without virtual, calling through a base pointer always calls the base version (static binding)
- Add the override keyword in derived classes to catch typos and signature mismatches at compile time

```
class Shape {
public:
    virtual double area() const { return 0; }
    virtual void draw() const { cout << "Drawing shape" << endl; }
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    void draw() const override { cout << "Drawing circle" << endl; }
};

class Square : public Shape {
    double s;
public:
    Square(double s) : s(s) {}
    double area() const override { return s * s; }
    void draw() const override { cout << "Drawing square" << endl; }
};
```

9.3 Base Class Pointers and Dynamic Dispatch

- Store derived objects as base class pointers -- allows generic processing of different object types
- At runtime the vtable (virtual dispatch table) routes the call to the correct override
- Always declare the base class destructor virtual when using polymorphism -- ensures the correct destructor chain

```
Shape* shapes[] = { new Circle(5), new Square(4), new Circle(3) };

for (Shape* s : shapes) {
    s->draw();
    cout << "Area = " << s->area() << endl;
}
// Each iteration calls the correct override based on actual type
```

9.4 Virtual Destructor

- Without a virtual destructor, deleting a derived object through a base pointer only calls the base destructor -- resource leak
- Always declare virtual ~ClassName() {} in any class that has virtual functions

```
class Base { public: virtual ~Base() { cout << "Base dtor" << endl; } };
class Derived : public Base { public: ~Derived() { cout << "Derived dtor" << endl; } }

Base* b = new Derived();
delete b; // prints: Derived dtor, then Base dtor
```